

SIMATS ENGINEERING

Department of Computer Science and Engineering

TECHNICAL REPORT

Resolution Algorithm Implementation for Propositional Logic Inference

Course Name	Artificial Intelligence
Course Code	CSA17
Assessment	Assessment Tool 3 – Annexure A
Course Outcome	CO3: Analyze knowledge representation and reasoning using propositional logic, FOL, inference, resolution and chaining methods (BL4)
Tool / Technique	Resolution Refutation Algorithm (Propositional Logic)
Language Used	Python 3

1. Objective

The objective of this assignment is to demonstrate mastery of propositional-logic knowledge representation and automated reasoning by applying the **Resolution Algorithm** to five real-world inference scenarios. For each scenario, the given facts and rules are formalized as propositional logic sentences, converted into Conjunctive Normal Form (CNF), and used — together with the negated goal — as the input to a resolution-refutation procedure that mechanically proves whether the goal is a logical consequence of the knowledge base.

2. Theoretical Background

2.1 Resolution Refutation

Resolution is a sound and complete inference rule for propositional (and first-order) logic. To prove that a knowledge base KB entails a goal G, the algorithm negates the goal, adds $\neg G$ to KB, and repeatedly applies the resolution rule to pairs of clauses containing complementary literals (a literal and its negation). If this process derives the **Empty Clause** (a contradiction, denoted NIL), then $KB \cup \{\neg G\}$ is unsatisfiable, which proves that KB logically entails G.

2.2 Conjunctive Normal Form (CNF)

All resolution operates on clauses (disjunctions of literals). Implications are converted using the equivalence $A \rightarrow B \equiv (\neg A \vee B)$, which is the only transformation required for the simple Horn-clause knowledge bases used in this assignment.

2.3 The Resolution Rule

Given two clauses $(\neg P \vee Q)$ and (P) , resolving on the complementary pair $\{P, \neg P\}$ yields the resolvent (Q) . Repeating this process — chaining derived facts through further implications — drives the proof toward the empty clause.

3. Methodology

- 1 Formalize each knowledge-base statement and the goal as propositional variables (P, Q, R, ...).
- 2 Convert every implication to CNF using $\neg A \vee B$.
- 3 Negate the goal and add it as an additional clause.
- 4 Represent each clause as a Python frozenset of literal strings (e.g. $\{\neg P, Q\}$), enabling fast set operations for complementary-literal detection.
- 5 Iterate over all pairs of clauses; whenever a pair shares complementary literals, compute the resolvent and add it to the clause set.
- 6 Terminate when the empty clause is produced (goal proved) or when no new clauses can be generated (goal not provable from the given KB).

4. Implementation Summary

The accompanying **resolution_algorithm.py** script implements a generic, reusable propositional-resolution engine (functions `negate_literal`, `resolve`, and `resolution_algorithm`) and applies it to all five knowledge bases from Annexure A. The engine prints the full clause set, every resolution step performed, and the final verdict for each question.

5. Results

Q#	Scenario	Goal	Resolution Steps	Result
1	Rain & Wet Ground	Ground is wet (Q)	2	PROVED (NIL)
2	Assignment Submission	Rahul gets marks (Q)	2	PROVED (NIL)
3	Library Membership	Priya can borrow (Q)	2	PROVED (NIL)
4	Placement Eligibility	Arun is eligible (Q)	2	PROVED (NIL)
5	Access Control	User granted access (R)	3	PROVED (NIL)

All five goals were successfully proved: the resolution procedure derived the Empty Clause (NIL) in every case, confirming that each conclusion follows logically and soundly from its respective knowledge base. Question 5 required one additional step because it involves a two-premise implication chain ($P \rightarrow Q \rightarrow R$) rather than a single implication.

5.1 Program Output (Screenshot)

Figure 1 below shows the first portion of the console output (Question 1). The complete output for all five questions is provided separately in **output.png**.

```
python3 resolution_algorithm.py - Terminal Output

=====
QUESTION 1 : RAIN AND WET GROUND
=====

Knowledge Base (Propositional Logic):
P = 'It rains'
Q = 'The ground is wet'
1. P -> Q      i.e. CNF: ( $\neg P \vee Q$ )
2. P          i.e. CNF: (P)

Goal: Q ('The ground is wet')
Negated Goal (added as a clause): ( $\neg Q$ )

Initial Clause Set (KB clauses + Negated Goal):
( $\neg Q$ )
(P)
( $Q \vee \neg P$ )

Step 1: Resolve ( $\neg Q$ ) and ( $Q \vee \neg P$ )
=> ( $\neg P$ )

Step 2: Resolve (P) and ( $Q \vee \neg P$ )
=> (Q)

Step 3: Resolve (P) and ( $\neg P$ )
=> {} <-- NIL (Empty Clause / contradiction)

-----
RESULT: Empty clause (NIL) derived -> GOAL IS PROVED TRUE.
-----

=====
QUESTION 2 : STUDENT ASSIGNMENT SUBMISSION
=====

Knowledge Base (Propositional Logic):
P = 'Student submits the assignment'
Q = 'Student receives internal marks'
1. P -> Q      i.e. CNF: ( $\neg P \vee Q$ )
2. P (Rahul submitted) i.e. CNF: (P)

Goal: Q ('Rahul receives internal marks')
Negated Goal (added as a clause): ( $\neg Q$ )

Initial Clause Set (KB clauses + Negated Goal):
( $\neg Q$ )
(P)
( $Q \vee \neg P$ )

Step 1: Resolve ( $\neg Q$ ) and ( $Q \vee \neg P$ )
=> ( $\neg P$ )
```

Figure 1: Terminal output of resolution_algorithm.py (Question 1 shown; see output.png for the full run covering all five questions).

6. Discussion

Questions 1–4 share an identical logical skeleton (a single Modus-Ponens-style implication $P \rightarrow Q$ with fact P), so each required exactly two resolution steps to reach NIL. Question 5 chains two implications ($P \rightarrow Q$ and $Q \rightarrow R$), so the algorithm first had to derive the intermediate fact Q before deriving R, taking three steps in total. This mirrors how forward chaining works in rule-based expert systems, and confirms that resolution subsumes chaining as a special case when the knowledge base consists of Horn clauses.

7. Conclusion

This assignment successfully demonstrates the complete pipeline of automated logical reasoning: translating natural-language statements into propositional logic, converting them to CNF, and

mechanically applying the Resolution Algorithm to prove each goal. The Python implementation generalizes cleanly across all five scenarios, validating both the theoretical soundness of resolution and its practical applicability to simple knowledge-based decision-making systems such as access-control and eligibility checks.

8. Files Submitted

- **Problem_Statement.pdf** – consolidated statement of all 5 questions.
- **Solution.pdf** – complete hand-worked resolution proofs for all 5 questions.
- **resolution_algorithm.py** – Python implementation of the generic resolution engine.
- **output.png** – screenshot of the program's console output.
- **Report.pdf** – this document.
- **README.md** – project overview and instructions to run the code.